



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

Konvertor tokova snaga: polarne koordinate

SEMINARSKI RAD

Student: Hasanić Azra

Broj indeksa: 19553

Predmet: Strukture i režimi rada elektroenergetskih sistema

Akadska godina: 2025/2026.

Sarajevo, 2026.

Sadržaj

1	Uvod	1
2	Proračun tokova snaga – teorijske osnove i matematički model	2
2.1	Formiranje matrice admitansi sistema	3
2.2	Jednačine balansa snaga u polarnim koordinatama	4
2.3	Numerički algoritam rješavanja	5
3	Korišteni softverski alati	6
3.1	Struktura MATPOWER (.m) datoteke	6
3.2	Struktura dTwin (.dmodl) datoteke	7
4	Implementacija i analiza programskog koda	9
4.1	Arhitektura i organizacija projekta	9
4.2	Analiza koda	9
4.2.1	Parsiranje MATPOWER datoteka	10
4.2.2	Formiranje elemenata grana i transformatora	10
4.2.3	Generisanje .dmodl jednačina sa polarnim koordinatama .	11
4.2.4	Multithread-ing i real-time indikator konverzije	12
4.3	GUI	12
4.4	Integracija u dTwin okruženje	13
4.5	CMake konfiguracija	14
5	Rezultati i testiranje programa	15
5.1	Samostalna aplikacija	15
5.2	dTwin plugin	18
5.3	Analiza i simulacija .dmodl file-a	20
6	Zaključak	27

1 Uvod

Proračun tokova snaga je jedan od najvažnijih aspekata analize elektroenergetskih sistema. Omogućava odredbu statičkog stanja mreže, to jeste proračun napona svih čvorova, faznih uglova napona, te tokova aktivnih i reaktivnih snaga kroz elemente mreže. Rezultantni podaci su neophodni za analizu rada sistema, razvoj i upravljanje istim.

U svrhu opisa i analize elektroenergetskih mreža moguće je koristiti različite software-ske alate među kojima je i MATPOWER, koji omogućava definisanje modela mreže i proračune pomoću MATLAB datoteka, koje sadrže podatke poput onih o sabirnicama, granama i generatorima mreže.

Okruženje dTwin, namijenjeno za modeliranje i simulaciju statičkih i dinamičkih sistema, koristi vlastiti format modela (.dmodl), te ne podržava direktnu upotrebu .m file-ova kao izvora podataka.

Budući da je neophodnu konverziju formata podataka nepraktično vršiti ručno u slučaju složenih sistema, razvijen je konvertor koji datoteke .m formata pretvara u odgovarajući .dmodl format, sa formiranim matematičkim modelom mreže spremnim za simulaciju i proračun.

Glavni cilj ovog rada jeste prikazati matematičko modeliranje elektroenergetskih sistema u polarnim koordinatama, te opisati implementaciju konvertora u obliku multithreaded plugin-a sa grafičkim interface-om za dTwin okruženje.

2 Proračun tokova snaga – teorijske osnove i matematički model

Proračun tokova snaga podrazumijeva rješavanje sistema nelinearnih algebarskih jednačina s ciljem odredbe stacionarnog stanja elektroenergetske mreže. Pri zadanim uvjetima proizvodnje i potrošnje energije, potrebno je odrediti napone na svim čvorovima mreže u formi modula i faznog ugla veličine, na osnovu čega je moguće računati tokove snaga kroz sve elemente sistema.[1]

U elektroenergetskom sistemu čvorovi se klasificiraju u tri osnovne grupe na osnovu poznatih i nepoznatih veličina:

- **Slack (balansni) čvor:** referentni čvor kojem su zadani magnituda napona V i fazni ugao δ (obično $\delta = 0^\circ$). Ovaj čvor balansira gubitke aktivne i reaktivne snage u mreži.
- **PV (generatorski) čvorovi:** Čvorovi u kojima su poznati aktivna snaga injeckiranja P i magnituda napona V . Traže se fazni ugao δ i reaktivna snaga Q .
- **PQ (potrošački) čvorovi:** Čvorovi u kojima su poznati aktivna i reaktivna snaga potrošnje (odnosno neto injeckiranja P i Q). Traže se magnituda napona V i fazni ugao δ .

Osnovni pojmovi i veličine korištene pri realizaciji proračuna:

Simbol	Značenje veličine	Jedinica
$P_{gen,i}$	Aktivna snaga proizvodnje u čvoru i	MW
$Q_{gen,i}$	Reaktivna snaga proizvodnje u čvoru i	MVAr
$P_{load,i}$	Aktivna snaga potrošnje u čvoru i	MW
$Q_{load,i}$	Reaktivna snaga potrošnje u čvoru i	MVAr
P_i^{net}	Neto injektirana aktivna snaga u čvoru i	p.u.
Q_i^{net}	Neto injektirana reaktivna snaga u čvoru i	p.u.
S_{base}	Bazna prividna snaga sistema	MVA
V_i	Modul (iznos) napona čvora i	p.u.
δ_i	Fazni ugao napona čvora i	rad, °
r	Otpornost grane (voda ili transformatora)	p.u.
x	Reaktansa grane (voda ili transformatora)	p.u.
b_{sh}	Ukupna shunt susceptansa grane	p.u.
y_{ik}	Serijska admitansa grane između čvorova i i k	p.u.
Y_{ii}	Sopstvena admitansa čvora i (dijagonalni element)	p.u.
Y_{ik}	Međusobna admitansa između čvorova i i k	p.u.
$ Y_{ik} $	Modul admitanse Y_{ik}	p.u.
φ_{ik}	Fazni ugao admitanse Y_{ik}	rad, °
tap	Modul regulacije prenosnog odnosa transformatora	—
ϕ	Ugao faznog pomaka transformatora	rad, °
τ	Kompleksni prenosni odnos transformatora	—
G_s, B_s	Shunt konduktansa i susceptansa u čvoru	MW, MVAr
ϵ	Tolerancija (kriterij konvergencije proračuna)	—

2.1 Formiranje matrice admitansi sistema

Osnova za formiranje jednačina balansa snaga jeste matrica admitansi sistema, označena kao Y_{bus} .

Za svaku granu između čvorova i i j definiše se serijska admitansa:

$$y_{ik} = \frac{1}{r + jx} = \frac{r}{r^2 + x^2} - j \frac{x}{r^2 + x^2} \quad (1)$$

Ukoliko grana posjeduje shunt susceptansu u vidu poput kapacitivnosti vodova, ona se prema ekvivalentnom π modelu polovi i dodaje na krajeve grane:

$$b_{sh,i} = b_{sh,k} = \frac{b_{sh}}{2} \quad (2)$$

U slučaju transformatora sa kompleksnim prenosnim odnosom $\tau = \text{tap}^* e^{j\Phi}$, gdje je tap magnituda regulacije, a Φ ugao faznog pomaka, elementi matrice admitancija se računaju na sljedeći način:

$$\tau = \text{tap} \cdot e^{j\phi} = \text{tap} \cdot \cos(\phi) + j \cdot \text{tap} \cdot \sin(\phi) \quad (3)$$

$$Y_{ii}^{\text{from}} = \frac{y_{ik} + j\frac{b_{sh}}{2}}{\tau \cdot \tau^*} \quad (4)$$

$$Y_{ii}^{\text{to}} = y_{ik} + j\frac{b_{sh}}{2} \quad (5)$$

$$Y_{ik} = \frac{-y_{ik}}{\tau^*}, Y_{ki} = \frac{-y_{ik}}{\tau} \quad (6)$$

Konačni dijagonalni i vandijagonalni elementi matrice Y_{bus} se formiraju sumiranjem svih grana povezanih sa čvorom i , uz dodatak sopstvene shunt admitancije čvora:

$$y_{sh,i} = \frac{G_s + jB_s}{S_{\text{base}}} \quad (7)$$

$$Y_{ii} = \sum_{k \neq i} Y_{ii}^{\text{grane}} + y_{sh,i} \quad (8)$$

$$Y_{ik} = -Y_{ik}^{\text{grane}} \quad (9)$$

2.2 Jednačine balansa snaga u polarnim koordinatama

U polarnim koordinatama, napon čvora i se predstavlja kao $Y_i = |Y_i| \angle \delta_i$, dok se elementi matrice admitancija predstavljaju preko svojih magnituda i uglova:

$$Y_{ik} = |Y_{ik}| \angle \varphi_{ik} \quad (10)$$

$$|Y_{ik}| = \sqrt{G_{ik}^2 + B_{ik}^2}, \varphi_{ik} = \arctan\left(\frac{B_{ik}}{G_{ik}}\right) \quad (11)$$

Razdvajanjem kompleksnog izraza za injektiranu prividnu snagu na imaginarni i realni dio, formiraju se jednačine balansa snaga u polarnom obliku, koje moraju biti zadovoljene na odgovarajućim čvorovima:

$$S_i = P_i + jQ_i = V_i \cdot I_i^* = V_i \sum_k (Y_{ik} V_k)^* \quad (12)$$

$$P_i = |Y_{ii}| V_i^2 \cos(\varphi_{ii}) + V_i \sum_{k \neq i} |Y_{ik}| V_k \cos(\delta_i - \varphi_{ik} - \delta_k) \quad (13)$$

$$Q_i = -|Y_{ii}| V_i^2 \sin(\varphi_{ii}) + V_i \sum_{k \neq i} |Y_{ik}| V_k \sin(\delta_i - \varphi_{ik} - \delta_k) \quad (14)$$

2.3 Numerički algoritam rješavanja

Konvertor kreiran u sklopu ovog rada ne rješava direktno ove jednačine, nego ih generiše unutar .dmodl datoteke, čime dTwin platforma dobija matematički model. Okruženje dTwin potom koristi Newton-Raphsonovu metodu za rješavanje sistema nelinearnih jednačina oblika $\mathbf{f}(\mathbf{x}) = 0$.

Iterativni postupak računa se vrši prema formuli:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{J}^{-1}\mathbf{f}(\mathbf{x}^{(k)}) \quad (15)$$

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{P}}{\partial \delta} & \frac{\partial \mathbf{P}}{\partial \mathbf{V}} \\ \frac{\partial \mathbf{Q}}{\partial \delta} & \frac{\partial \mathbf{Q}}{\partial \mathbf{V}} \end{bmatrix} \quad (16)$$

Za ostvarivanje konvergencije pri proračunu potrebna je dobra aproksimacija početnih vrijednosti, zbog čega je korištena “Flat start” inicijalizacija vrijednosti napona uglova, te kriterij konvergencije vrijednosti $\epsilon = 10^{-6}$.

3 Korišteni softverski alati

Za implementaciju ovog projekta odabran je Visual Studio IDE koji omogućava debugging, praćenje memorije kao i rada sa više niti (thread-ova) u realnom vremenu. Budući da je jedan od glavnih zahtjeva projekta bio da bude funkcionalan na različitim operativnim sistemima, kreiran i konfigurisan je kroz CMake, koji je dodatno omogućio upotrebu biblioteka i funkcija iz natID repozitorija i paketa[2].

3.1 Struktura MATPOWER (.m) datoteke

Za definisanje energetske mreže, MATPOWER [4] koristi standardne MATLAB skripte. Podaci su organizovani u obliku globalnih matrica, od kojih su za proračun tokova snaga ključne tri: bus, gen i branch, uz definisanje bazne snage sistema preko varijable mpc.baseMVA.

Svaki red u **matrici čvorova** predstavlja jedan čvor (sabirnicu) u sistemu:

- **Kolona 1** (*bus_i*): Jedinstveni identifikacioni broj čvora (ID).
- **Kolona 2** (*type*): Tip čvora (1 = PQ čvor, 2 = PV čvor, 3 = Slack čvor).
- **Kolona 3** (*Pd*): Aktivna snaga potrošnje u čvoru (MW).
- **Kolona 4** (*Qd*): Reaktivna snaga potrošnje u čvoru (MVar).
- **Kolona 5** (*Gs*), **Kolona 6** (*Bs*): Konduktansa i susceptansa shunta čvora (MW i MVar pri naponu od 1.0 p.u.).
- **Kolona 8** (*Vm*): Početni modul napona (p.u.).
- **Kolona 9** (*Va*): Početni fazni ugao napona (u stepenima).

Matrica generatora (mpc.gen):

- **Kolona 1** (*bus*): ID čvora na koji je generator priključen.
- **Kolona 2** (*Pg*): Izlazna aktivna snaga generatora (MW).
- **Kolona 3** (*Qg*): Izlazna reaktivna snaga generatora (MVar).
- **Kolona 6** (*Vg*): Zadani (kontrolisani) napon generatora (p.u.).

Matrica grana (mpc.branch) definiše prenosne vodove i transformatore koji spajaju čvorove:

- **Kolona 1** (*f_bus*), **Kolona 2** (*t_bus*): ID početnog ("from") i krajnjeg ("to") čvora.
- **Kolona 3** (*r*): Otpornost grane (p.u.).
- **Kolona 4** (*x*): Reaktansa grane (p.u.).

- **Kolona 5 (*b*):** Ukupna susceptansa punjenja vodova (p.u.).
- **Kolona 9 (*tap*):** Prenosni odnos (izvod) transformatora. Ako je vrijednost 0 ili 1, grana se tretira kao obični vod. Ako je različita od 1, riječ je o transformatoru sa promjenljivim izvodom.
- **Kolona 10 (*shift*):** Ugao faznog pomaka transformatora (u stepenima).

3.2 Struktura dTwin (.dmodl) datoteke

dTwin platforma [3] koristi .dmodl format zasnovan na deklarativnom opisivanju sistema nelinearnih jednačina (NLE). Datoteka se sastoji iz nekoliko dijelova koje konvertor mora sekvencijalno generisati.

Zaglavlje modela

Model se otvara deklaracijom tipa, domena i tolerancije rješavanja. konvertor ovdje eksplicitno postavlja parametre nelinearnog solvera:

```
Model [type=NL domain=real eps=1e-6 name="PowerFlow_Polar"]:
```

Sekcija varijabli (Variables)

Za svaki čvor ID iz MATPOWER datoteke, konvertor generiše parove varijabli za napon i ugao:

- V_ID : Modul (iznos) napona čvora.
- δ_ID : Fazni ugao napona čvora.

Takoder, definisani su i fiksni parametri elemenata matrice admitansi, kao što su npr. $Y_mag_i_k$ i $Y_ang_i_k$ za modul i ugao admitanse između čvorova i i k .

Sekcija inicijalizacije (Init)

Ovdje se postavlja tzv. "Flat Start" ili početne vrijednosti iz .m datoteke kako bi Newton-Raphson algoritam mogao konvergirati. Konvertor ovdje upisuje:

```
Init:
    delta_1 = 0; V_1 = 1.05
    delta_2 = 0; V_2 = 1.0
```

Jednačine (Equations)

Umjesto matrica, ovdje se upisuju eksplicitne tekstualne jednačine balansa snaga u polarnom obliku. Svaki red predstavlja jednu jednačinu koja se mora zadovoljiti (lijeva strana *lhs* jednaka desnoj strani *rhs*).

Transformacija i mapiranja podataka

konvertor vrši transformaciju podataka u 3 koraka:

- *Normalizacija*: Vrijednosti snaga iz .m datoteke koje su zadate u MW i MVAr konvertor dijeli sa baznom snagom **baseMVA** i pretvara ih u jedinice (p.u.) koje dTwin zahtijeva.

- *Topološka transformacija:* Konvertor čita matricu grana i za svaki čvor kreira listu povezanih čvorova.
- *Prevođenje u tekstualne jednačine:* Za svaki čvor se generiše tekstualni niz (string) koji predstavlja formulu balansa snaga spremnu za dTwin solver.

4 Implementacija i analiza programskog koda

U ovom poglavlju je analizirana softverska arhitektura projekta, organizacija izvornih datoteka, te integracija u funkcionalan plugin za dTwin platformu.

4.1 Arhitektura i organizacija projekta

Softverski sistem je podijeljen na dva zasebna projekta kako bi se omogućilo odvojeno testiranje i lakša integracija:

- **PowerFlowConverter:** Samostalna (*standalone*) aplikacija koja se kompajlira u izvršni `.exe` fajl. Primarno služi za interno testiranje, verifikaciju algoritama i otklanjanje grešaka van dTwin okruženja.
- **PowerFlowPlugin:** Dinamička biblioteka (`.dll`) namijenjena za direktnu integraciju u dTwin okruženje kao izvorni dodatak (*plugin*).

Struktura direktorija i organizacija izvornog koda prikazana je u nastavku:

```
PowerFlowConverter/  
  CMakeLists.txt  
  powerFlowConverter.cmake  
  res/  
    DevRes.xml  
    main.xml  
  tr/  
    EN/main.xml  
    BA/main.xml  
  src/  
    Converter.h  
    main.cpp  
    Application.h  
    MainWindow.h  
    MainView.h  
    ViewInput.h  
    ViewProgress.h  
    ViewResults.h  
    MenuBar.h / ToolBar.h  
    Info.plist
```

4.2 Analiza koda

Zbog složenosti i dužine koda, u ovom potpoglavlju će biti analizirani samo najznačajniji dijelovi koda, koji predstavljaju suštinu procesa.

4.2.1 Parsiranje MATPOWER datoteka

Kako bi se osiguralo isključivo numeričke vrijednosti za dalji rad unutar dTwin modela, neophodno je sve izraze parsirati i konvertovati u slučaju da su izvorno dostupni u formi razlomaka, korijena i sličnih izraza. U tu svrhu, unutar klase Converter, u funkciji evalExpr() implementiran je evaluator izraza:

```
// Primjer provjere i parsiranja izraza iz koda
static double evalExpr(const std::string& raw) {
    char* end = nullptr;
    double v = std::strtod(raw.c_str(), &end);
    if (end && *end == '\0') return v; // Ako je čist broj,
    vrati ga odmah

    // U suprotnom, aktivira se rekurzivni evaluator za
    složene izraze (npr. sa sqrt)
    Eval e; e.p = raw.c_str();
    return e.expr();
}
```

Tokom parsiranja tekstualnih matrica mpc.bus, mpc.gen i mpc.branch, uklanjaju se MATLAB komentari (%) i razlučuju granice redova i kolona.

4.2.2 Formiranje elemenata grana i transformatora

Nakon što se iz mpc.branch sekcije pročitaju parametri grana (otpornost r, reaktansa x i šant susceptansa b), program pristupa izgradnji matrice admitanse sistema prema formulama navedenim u prethodnim poglavljima.

```
double r = row[2], x = row[3], b = row[4];
double tap = row[8]; double angDeg = row[9];

// Proračun serijske admitanse y = 1 / (r + jx)
Cmplx y = (r == 0 && x == 0) ? Cmplx(0,0) : Cmplx(1.0) /
    Cmplx(r, x);
Cmplx bsh(0.0, b / 2.0); // Polovina shunt susceptanse

// Korekcija za transformatore sa tapom i faznim pomakom
double angRad = angDeg * M_PI / 180.0;
Cmplx a(tap * std::cos(angRad), tap * std::sin(angRad)); //
    tau = tap * e^(jphi)

Cmplx dii_f = (y + bsh) / (a * std::conj(a)); // Y_ii na "
    from" strani
Cmplx dii_t = y + bsh; // Y_ii na "
    to" strani
Cmplx dij = -y / std::conj(a); // Y_ik
    element
Cmplx dji = -y / a; // Y_ki
    element
```

Budući da su matrice admitansi realnih sistema izrazito rijetke (većina elemenata izvan dijagonale su nule), u kodu je korištena biblioteka za rijetke matrice `matID sparse` i objekat `sparse::CmplxSolverReleaser`. Ovakva implementacija smanjuje količinu zauzete memorije i vrijeme izvršavanja koda.

```
int estNZ = n + 2 * nBranches + 16; // Procjena broja
    nenultih elemenata
sparse::CmplxSolverReleaser pYSolver(
    sparse::createCmplxSolver(n, estNZ, sparse::Symmetry::
        NonSymmetric, sparse::SolverType::LU)
);
// Punjenje sparse solvera elementima
pYSolver->addTriple(fi, fi, dii_f);
pYSolver->addTriple(fi, ti, dij);

// Optimizirani pristup vandijagonalnim elementima
static int64_t yijKey(int i, int j) {
    return (int64_t)i * 1000000 + j;
}
```

4.2.3 Generisanje .dmodl jednačina sa polarnim koordinatama

Prilikom upisa modela u izlaznu .dmodl datoteku, vrši se konverzija elemenata Ybus u polarni oblik pomoću funkcija `std::abs()` i `std::arg()`:

```
double mag = std::abs(yii); // Modul |Y_ik|
double ang = std::arg(yii); // Ugao phi_ik u radijanima
```

Nakon toga, generiše se tekstualni string koji predstavlja nelinearnu jednačinu balansa snaga za svaki čvor:

```
// Dijagonalni element: |Y_ii| * V_i^2 * cos(phi_ii)
std::string lhs = YM + "_" + std::to_string(bid) + "_" + std::
    to_string(bid) +
        "*" + V + "_" + std::to_string(bid) + "^2*
    cos(" +
        YA + "_" + std::to_string(bid) + "_" + std::
    to_string(bid) + ")";

// Vandijagonalna suma po susjednim čvorovima: V_i * sum /
    |Y_ik| * V_k * cos(delta_i - phi_ik - delta_k)
lhs += " + " + V + "_" + std::to_string(bid) + " * (\n";
for (int j : adj) {
    int bj = idxToId[j];
    std::string term = YM + "_" + std::to_string(bid) + "_"
        + std::to_string(bj) +
        "*" + V + "_" + std::to_string(bj) +
        "*cos(" + D + "_" + std::to_string(
        bid) +
```

```

        "-" + YA + "-" + std::to_string(bid)
    + "-" + std::to_string(bj) +
        "-" + D + "-" + std::to_string(bj) +
    ");
    lhs += first ? term : " + " + term;
    first = false;
}

```

4.2.4 Multithread-ing i real-time indikator konverzije

Proces konverzije složenih mreža može trajati duže što bi u slučaju standardne arhitekture uzrokovalo blokiranje rada korisničkog interfejsa. Kako bi se to izbjeglo, implementirani program je podijeljen u 2 niti (threads). Radnu nit koja izvršava pozadinsku metodu `Converter::convert()`, obavlja parsiranje, gradi Ybus matricu i generiše .dmodl datoteku, te korisničku nit koja upravlja prozorima i grafičkim elementima. Unutar `MainWindow` klase instanciran je periodični tajmer (`td::Timer`) koji se okida svakih 100 ms.

Radna nit asinhrono inkrementira `progressPercent`, dok GUI nit preko tajmera čita to stanje i osvježava Progress Bar. Za prenos finalnih rezultata ili poruka o greškama iz radne niti nazad u glavnu nit koristi se funkcija `gui::thread::asyncExecInMainThread()`.

```

std::atomic<int> progressPercent{ 0 };
std::atomic<bool> stopRequested{ false };

```

4.3 GUI

Grafički korisnički interfejs je realizovan upotrebom `natID` GUI funkcija, te XML definisanjem izgleda prozora.

U samostalnoj (*standalone*) aplikaciji, centralni prozor `MainWindow` sadrži kontejner `MainView` koji je podijeljen na tri tab-a:

- **ViewInput:** Sadrži kontrole za odabir ulaznog .m file-a i izlazne putanje za .dmodl, kao i napredne opcije za konfiguraciju proračuna:
 - *Flat Start:* Postavlja početne vrijednosti napona svih čvorova na 1.0 p.u. i uglove na 0°, zanemarujući početne procjene iz datoteke.
 - *Zero Loads:* Privremeno postavlja aktivnu i reaktivnu snagu svih potrošača na nulu, što omogućava testiranje ponašanja mreže u režimu praznog hoda.
 - *Naponska ograničenja:* Aktivira kontrolu i provjeru minimalnih i maksimalnih dozvoljenih vrijednosti napona na sabirnicama tokom proračuna.
- **ViewProgress:** Prikazuje grafički *progress bar* i tekstualni log prozor sa detaljnim ispisom faza konverzije u realnom vremenu.

- **ViewResults:** Prikazuje konačnu statistiku mreže nakon završetka rada algoritma (ukupan broj čvorova, grana i proizvodnih kapaciteta).

U *plugin* verziji, interface je optimizovan unutar klase `WindowPlugin` i koristi dinamički `TabView` prilagođen `dTwin` radnom prostoru.

4.4 Integracija u dTwin okruženje

Integracija i izvršavanje *plugin*-a unutar `dTwin` okruženja se vrši sljedećim postupkom:

1. Kompajlirani `.dll` file se smješta u direktorij aplikacije: `ba.natID/dTwin/plugins/`.
2. `dTwin` prilikom pokretanja skenira ovaj direktorij, detektuje biblioteku, učitava je u memoriju i poziva eksportovanu funkciju `getPluginInterface()`.
3. Dodatak se registruje i pojavljuje u glavnom meniju `dTwin`-a pod opcijom **Model** → **Import** → **Konvertor tokova snaga: Polarne koordinate**, kao što je prikazano na slici 1.
4. Kada korisnik pokrene konverziju i pozadinska radna nit završi generisanje, aktivira se povratna funkcija `_onComplete()`. Ona šalje signal `dTwin`-u da je konverzija uspješna, nakon čega `dTwin` automatski preuzima novi kreirani `.dmodl` fajl, učitava ga u svoj radni prostor i generiše matematički model za simulaciju.

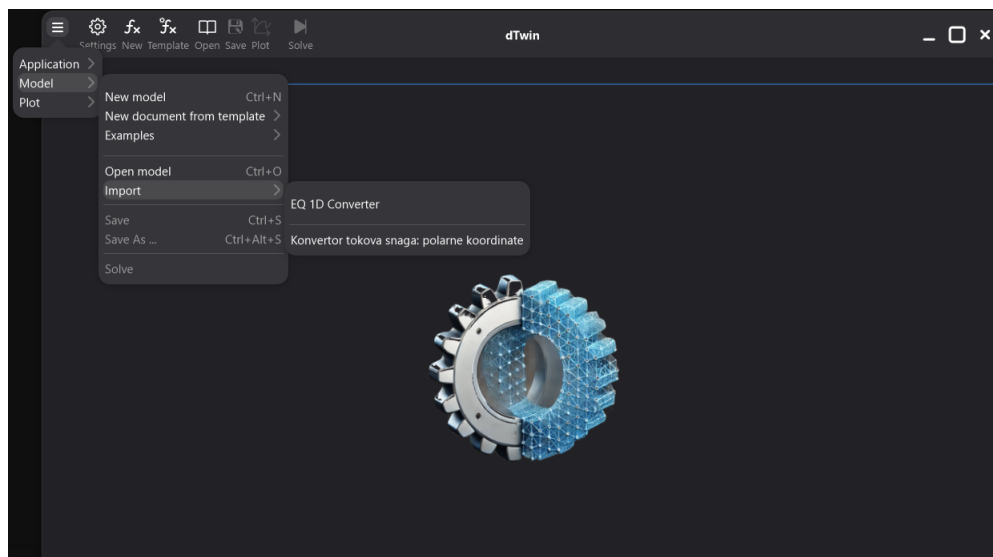


Figure 1: Prikaz dostupnog plugin-a u `dTwin` okruženju

4.5 CMake konfiguracija

Kako bi program djelovao kao dinamička biblioteka umjesto izvršnog programa, unutar CMakeLists.txt datoteke koristi se komanda `add_library` sa flag oznakom `SHARED`:

```
add_library(${PROJECT_NAME} SHARED ${SOURCE_FILES})
```

Za osiguravanje kompatibilnosti programa sa više operativnih sistema implementirana je sljedeća uslovna konfiguracija:

```
if(WIN32)
    target_compile_definitions(${PROJECT_NAME} PRIVATE
        WINDOWS PLUGIN_EXPORTS)
elseif(APPLE)
    target_compile_definitions(${PROJECT_NAME} PRIVATE MACOS
    )
else()
    target_compile_definitions(${PROJECT_NAME} PRIVATE LINUX
    )
endif()
```

Kako bi funkcija `getPluginInterface()` bila ulazna tačka za dTwin, definisan je `PLUGIN_EXPORTS` koji na osnovu CMake definicija upravlja specifikatorima eksporta simbola zavisno od ciljanog operativnog sistema:

```
#ifdef WINDOWS
    #ifdef PLUGIN_EXPORTS
        #define PLUGIN_API __declspec(dllexport)
    #else
        #define PLUGIN_API __declspec(dllimport)
    #endif
#else
    #define PLUGIN_API __attribute__((visibility("default")))
)
#endif
```

`powerFlowConverter.cmake` skripta omogućava automatsko povezivanje sa eksternim bibliotekama unutar `natID` sistema, prvenstveno sa `natGUI` i `MatrixLib` (za operacije sa matricama i vektorima). Implementiranjem ovog CMake pristupa, cijeli build sistem je u potpunosti automatizovan i nezavisan od platforme.

5 Rezultati i testiranje programa

U ovom poglavlju je prikazan rad razvijenog softvera kao samostalne aplikacije i kao integrisanog dijela dTwin okruženja.

5.1 Samostalna aplikacija

Prije integracije u dTwin platformu, konvertor je razvijan i testiran kao “standalone” aplikacija kako bi se potvrdila tačnost parsera i algoritma za formiranje matrice admitansi.

Pomoću alata CMake generisan je projektni fajl kompatibilan sa Visual Studio IDE odabirom odgovarajuće putanje foldera, te koracima Configure, Generate i Open project respektivno, kao što je prikazano na slici 2.

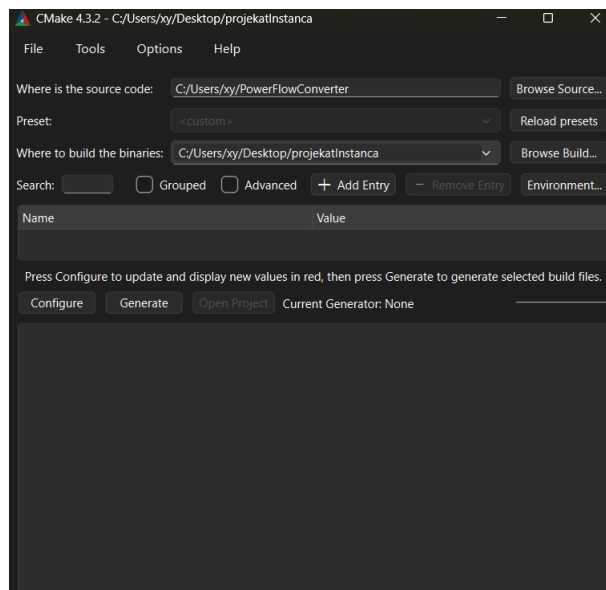


Figure 2: Prikaz procesa CMake konfiguracije i generisanja projekta

Po otvaranju projekta unutar Visual Studio okruženja, isti je potrebno postaviti kao “Startup Project”, te ga pokrenuti kompajliranjem njegovog “main” koda kao na slici 3.

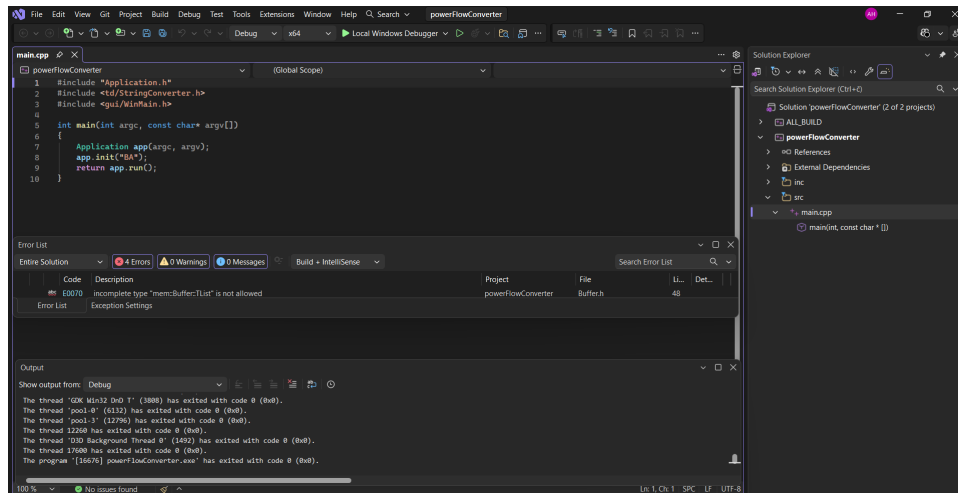


Figure 3: Prikaz pokretanja procesa unutar Visual Studio okruženja

Kompajliranjem koda projekta, dobija se samostalna aplikacija sa 3 odgovarajuća prozora za pokretanje konverzije, praćenje toka procesa te pojednostavljeni pregled rezultata konverzije. Rezultantni .dmodl file se automatski kreira i otvara, te će njegova struktura biti pojašnjena u narednom potpoglavlju.

Slika 4 prikazuje glavni tab korisničkog interface-a, koji omogućava unos željene .m datoteke pretragom file-ova na korisnikovom računaru, odabir dodatnih uvjeta konverzije, te pokretanje i zaustavljanje iste.

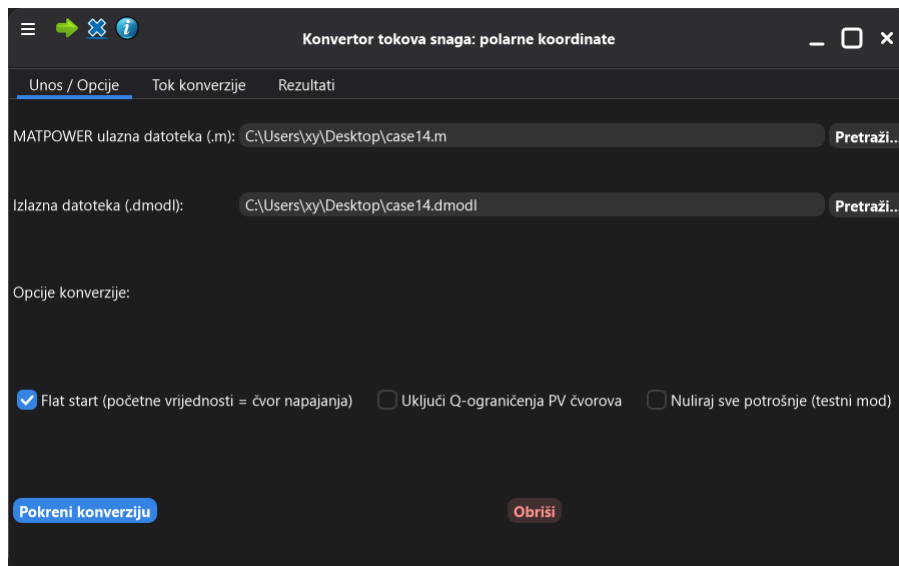


Figure 4: Glavni tab samostalne aplikacije konvertora

Na slikama 5 i 6 prikazani su tab-ovi kojima je moguće pratiti tok konverzije tekstualno kao i kroz real-time progress bar, te numeričke podatke o analiziranoj energetske mreži, poput broja i vrste čvorova, grana i generatora.

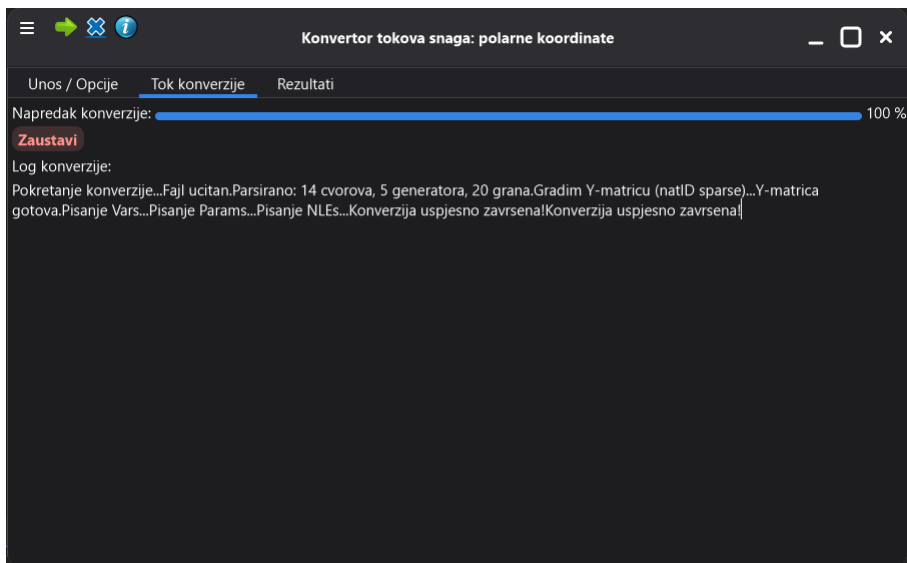


Figure 5: Tab za prikaz i praćenje toka konverzije

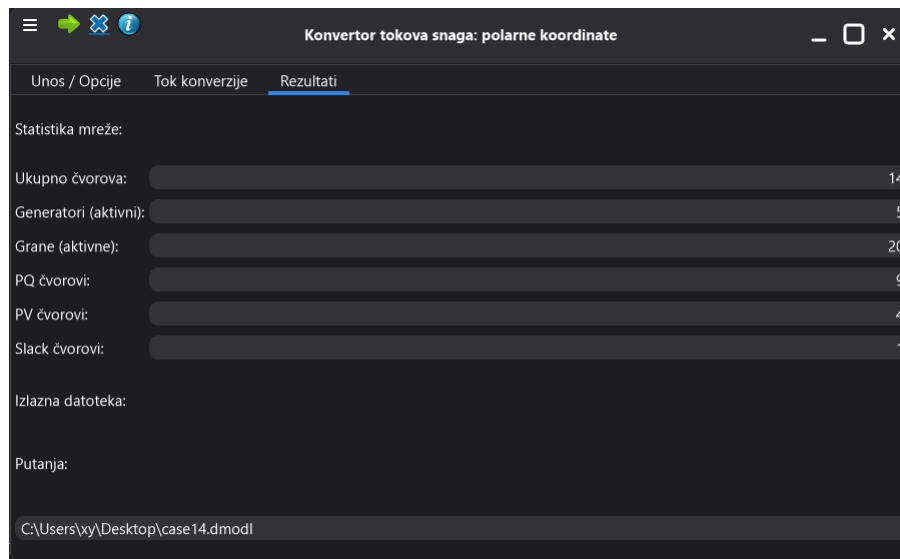


Figure 6: Tab numeričkih rezultata konverzije

5.2 dTwin plugin

Nakon verifikacije rada, kod je kompajliran u formi dinamičke biblioteke (*.dll* za Windows okruženje) i pozstavljen u dTwin direktorij namijenjen za vanjske dodatke (plugins). Platforma dTwin prepoznaje i učitava biblioteku, otvarajući GUI prikazan na slici 8, sa sljedećim opcijama:

- **Input File (.m):** Polje za odabir ulazne MATPOWER datoteke koja sadrži podatke o mreži.
- **Output File (.dmodl):** Definisanje lokacije i naziva izlaznog fajla u kojem će se generirati dTwin model.
- **Konfiguracioni parametri:** Korisnik može uključiti ili isključiti "Flat Start" inicijalizaciju, uzimanje u obzir reaktivnih ograničenja generatora, te predefinisanje nultih opterećenja.
- **Kontrole:** Dugme "Convert" pokreće radnu nit za konverziju, dok dugme "Cancel" sigurno prekida proces.

Na slici 7 prikazan je način pristupa i otvaranja plugin-a unutar dTwin-a.

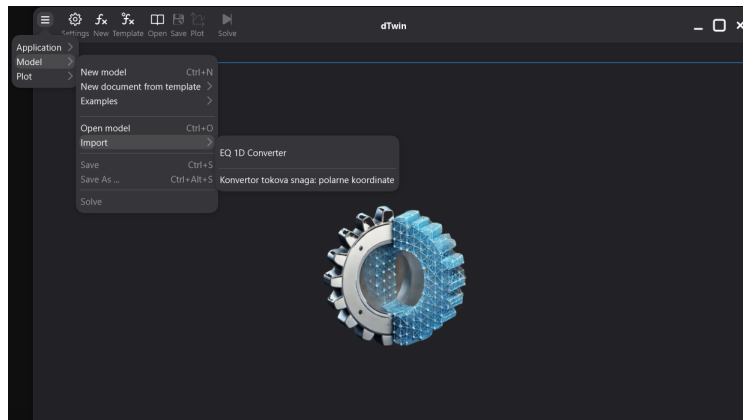


Figure 7: Pristup plugin-u unutar dTwin okruženja

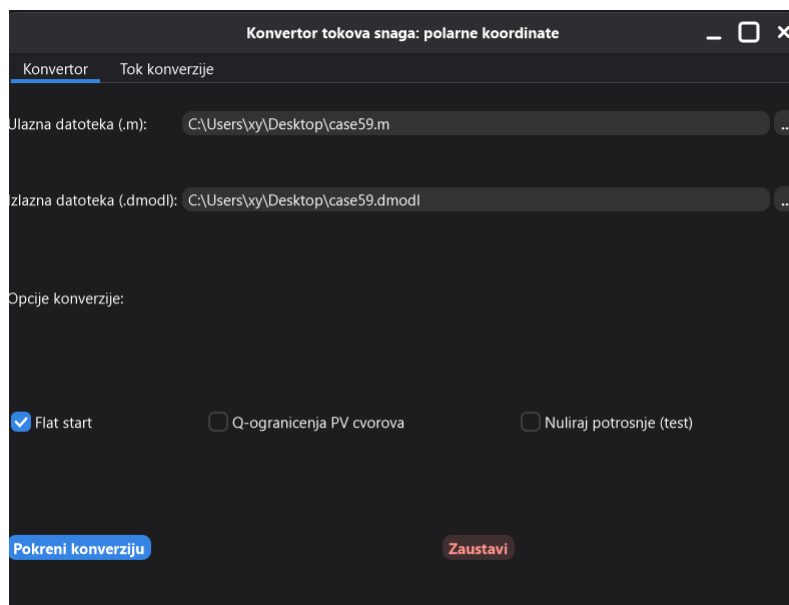


Figure 8: Glavni tab konvertora

Tok konverzije sa tekstualnim praćenjem i progress bar vizualizacijom je prikazan na slici 9, te se nakon izvršavanja konverzije kreira i otvara odgovarajući .dmodl file.

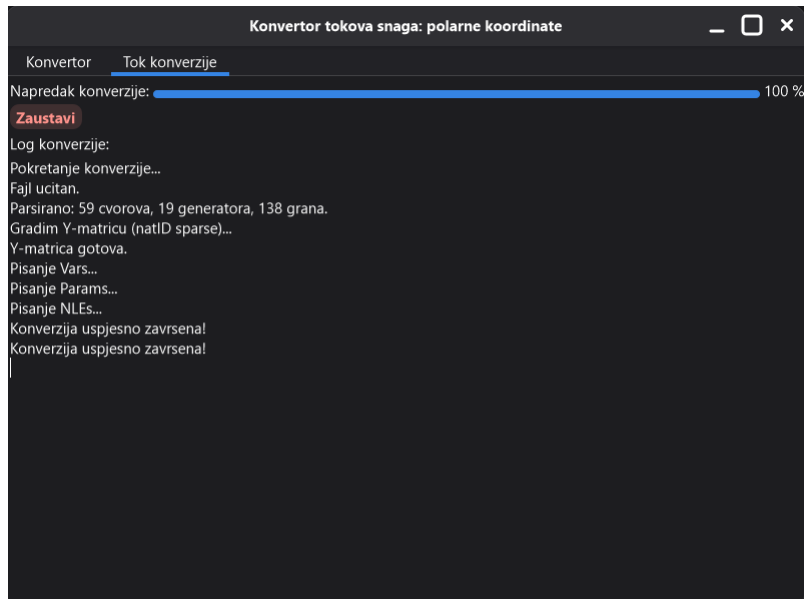


Figure 9: Tab za praćenje toka konverzije

5.3 Analiza i simulacija .dmodl file-a

Kreirani .dmodl file po otvaranju izgleda kao na slici 10, te sadrži parametre mreže i odgovarajuće jednačine balansa snaga. Pokretanje modela opcijom “Solve” rezultira proračunom i ispisom svih napona čvorova mreže, u formi modula i ugla faznog pomaka veličine, u donjem “Res” panelu vidljivom na slici 11.

```

1//Generisano MATPOWER Power Flow Converterom (polarne koordinate)
2Model [type=nl domain=real eps=1e-6 name="PF in polar coordinates"]:
3Vars [out=true]:
4  delta_2 = delta_1; V_2 = V_1
5  delta_3 = delta_1; V_3 = V_1
6  delta_4 = delta_1; V_4 = V_1
7  delta_5 = delta_1; V_5 = V_1
8  delta_6 = delta_1; V_6 = V_1
9  delta_7 = delta_1; V_7 = V_1
10 delta_8 = delta_1; V_8 = V_1
11 delta_9 = delta_1; V_9 = V_1
12 delta_10 = delta_1; V_10 = V_1
13 delta_11 = delta_1; V_11 = V_1
14 delta_12 = delta_1; V_12 = V_1
15 delta_13 = delta_1; V_13 = V_1
16 delta_14 = delta_1; V_14 = V_1
17 delta_15 = delta_1; V_15 = V_1
18 delta_16 = delta_1; V_16 = V_1
19 delta_17 = delta_1; V_17 = V_1
20 delta_18 = delta_1; V_18 = V_1
21 delta_19 = delta_1; V_19 = V_1
22 delta_20 = delta_1; V_20 = V_1
23 delta_21 = delta_1; V_21 = V_1
24 delta_22 = delta_1; V_22 = V_1
25 delta_23 = delta_1; V_23 = V_1
26 delta_24 = delta_1; V_24 = V_1
27 delta_25 = delta_1; V_25 = V_1
28 delta_26 = delta_1; V_26 = V_1
29 delta_27 = delta_1; V_27 = V_1
30 delta_28 = delta_1; V_28 = V_1
31 delta_29 = delta_1; V_29 = V_1
32 delta_30 = delta_1; V_30 = V_1
33 delta_31 = delta_1; V_31 = V_1

```

Figure 10: Kreirani dTwin model mreže

repeat	outiter	basel	totiter	info	eps	delta_2	V_2	delta_3	V_3	delta_4	V_4	delta_5	V_5	delta_6	V_6
1	0	1	5	5	6.82e-11	-0.026930	1.523398	0.880304	1.000000	0.684627	1.000000	0.582950	1.000000	0.707703	1.000000

Figure 11: Tab za praćenje toka konverzije

Testiranje programa je moguće vršiti na različitim modelima mreža, dok je u nastavku prikazan primjer koda .dmodl file-a, dobijenog konverzijom case14.m:

```

Model [type=NL domain=real eps=1e-6 name="PF in polar
coordinates"]:
Vars [out=true]:
    delta_2 = delta_1; V_2 = V_1
    delta_3 = delta_1; V_3 = V_1
    delta_4 = delta_1; V_4 = V_1
    delta_5 = delta_1; V_5 = V_1
    delta_6 = delta_1; V_6 = V_1
    delta_7 = delta_1; V_7 = V_1
    delta_8 = delta_1; V_8 = V_1
    delta_9 = delta_1; V_9 = V_1
    delta_10 = delta_1; V_10 = V_1
    delta_11 = delta_1; V_11 = V_1
    delta_12 = delta_1; V_12 = V_1
    delta_13 = delta_1; V_13 = V_1
    delta_14 = delta_1; V_14 = V_1
Params:
    delta_1 = 0 [out=true]; V_1 = 1.06 [out=true]
    Y_1_1 = 20.3590155631616; phiY_1_1 =
-1.27035780366569

```

```

Y_1_2 = 16.0609192444319; phiY_1_2 =
1.88731558145995
Y_1_5 = 4.35747086940998; phiY_1_5 =
1.80846157863521
Y_2_2 = 31.7341546920484; phiY_2_2 =
-1.26606808897049
Y_2_1 = 16.0609192444319; phiY_2_1 =
1.88731558145995
Y_2_3 = 4.91472112830875; phiY_2_3 =
1.80384283722427
Y_2_4 = 5.3865118176269; phiY_2_4 = 1.88915717914878
Y_2_5 = 5.46541471274985; phiY_2_5 = 1.8873100601241
Y_3_3 = 10.3062971325906; phiY_3_3 =
-1.26314185340885
Y_3_2 = 4.91472112830875; phiY_3_2 =
1.80384283722427
Y_3_4 = 5.44398797507477; phiY_3_4 =
1.94421607304359
Y_4_4 = 40.058306260219; phiY_4_4 =
-1.30524409029666
Y_4_2 = 5.3865118176269; phiY_4_2 = 1.88915717914878
Y_4_3 = 5.44398797507477; phiY_4_3 =
1.94421607304359
Y_4_5 = 22.6369831989984; phiY_4_5 =
1.87779995488015
Y_4_7 = 4.88951266031734; phiY_4_7 = 1.5707963267949
Y_4_9 = 1.8554995578159; phiY_4_9 = 1.5707963267949
Y_5_5 = 36.7992730539438; phiY_5_5 =
-1.30776830434821
Y_5_1 = 4.35747086940998; phiY_5_1 =
1.80846157863521
Y_5_2 = 5.46541471274985; phiY_5_2 = 1.8873100601241
Y_5_4 = 22.6369831989984; phiY_5_4 =
1.87779995488015
Y_5_6 = 4.25744533525338; phiY_5_6 = 1.5707963267949
Y_6_6 = 18.547140114668; phiY_6_6 =
-1.20813089588316
Y_6_5 = 4.25744533525338; phiY_6_5 = 1.5707963267949
Y_6_11 = 4.53691320382116; phiY_6_11 =
2.0163039557173
Y_6_12 = 3.52353852490387; phiY_6_12 =
2.01870129996994
Y_6_13 = 6.84448501457259; phiY_6_13 =
2.04065769769006
Y_7_7 = 19.5490059482647; phiY_7_7 =
-1.5707963267949
Y_7_4 = 4.88951266031734; phiY_7_4 = 1.5707963267949
Y_7_8 = 5.67697984672154; phiY_7_8 = 1.5707963267949
Y_7_9 = 9.09008271975275; phiY_7_9 = 1.5707963267949
Y_8_8 = 5.67697984672154; phiY_8_8 =

```



```

-1.5707963267949
    Y_8_7 = 5.67697984672154; phiY_8_7 = 1.5707963267949
    Y_9_9 = 24.6741914908221; phiY_9_9 =
-1.35322863005945
    Y_9_4 = 1.8554995578159; phiY_9_4 = 1.5707963267949
    Y_9_7 = 9.09008271975275; phiY_9_7 = 1.5707963267949
    Y_9_10 = 11.0755309633021; phiY_9_10 =
1.93083736439992
    Y_9_14 = 3.34707906953128; phiY_9_14 =
2.01025233051656
    Y_10_10 = 15.8602059515102; phiY_10_10 =
-1.19757259820288
    Y_10_9 = 11.0755309633021; phiY_10_9 =
1.93083736439992
    Y_10_11 = 4.78786394100919; phiY_10_11 =
1.97451877297019
    Y_11_11 = 9.32274355859266; phiY_11_11 =
-1.1467436376609
    Y_11_6 = 4.53691320382116; phiY_11_6 =
2.0163039557173
    Y_11_10 = 4.78786394100919; phiY_11_10 =
1.97451877297019
    Y_12_12 = 6.75149452557881; phiY_12_12 =
-0.93392632180126
    Y_12_6 = 3.52353852490387; phiY_12_6 =
2.01870129996994
    Y_12_13 = 3.35658056818654; phiY_12_13 =
2.40615288591429
    Y_13_13 = 12.6121870085797; phiY_13_13 =
-1.00840569662484
    Y_13_6 = 6.84448501457259; phiY_13_6 =
2.04065769769006
    Y_13_12 = 3.35658056818654; phiY_13_12 =
2.40615288591429
    Y_13_14 = 2.57911062305547; phiY_13_14 =
2.02733884987258
    Y_14_14 = 5.92597705754879; phiY_14_14 =
-1.12390420156478
    Y_14_9 = 3.34707906953128; phiY_14_9 =
2.01025233051656
    Y_14_13 = 2.57911062305547; phiY_14_13 =
2.02733884987258
    P_2_g = 0.183
    P_3_g = -0.942
    P_4 = -0.478
    Q_4 = 0.039
    P_5 = -0.076
    Q_5 = -0.016
    P_6_g = -0.112
    P_8_g = 0

```

```

P_9 = -0.295
Q_9 = -0.166
P_10 = -0.09
Q_10 = -0.058
P_11 = -0.035
Q_11 = -0.018
P_12 = -0.061
Q_12 = -0.016
P_13 = -0.135
Q_13 = -0.058
P_14 = -0.149
Q_14 = -0.05
V_2_sp = 1.045
V_3_sp = 1.01
V_6_sp = 1.07
V_8_sp = 1.09

```

NLEs :

```

Y_2_2*V_2^2*cos(phiY_2_2) + V_2 * (Y_2_1*V_1*cos(
delta_2-phiY_2_1-delta_1) + Y_2_3*V_3*cos(delta_2-
phiY_2_3-delta_3) + Y_2_4*V_4*cos(delta_2-phiY_2_4-
delta_4) + Y_2_5*V_5*cos(delta_2-phiY_2_5-delta_5)) =
P_2_g
V_2=V_2_sp
Y_3_3*V_3^2*cos(phiY_3_3) + V_3 * (Y_3_2*V_2*cos(
delta_3-phiY_3_2-delta_2) + Y_3_4*V_4*cos(delta_3-
phiY_3_4-delta_4)) = P_3_g
V_3=V_3_sp
Y_4_4*V_4^2*cos(phiY_4_4) + V_4 * (Y_4_2*V_2*cos(
delta_4-phiY_4_2-delta_2) + Y_4_3*V_3*cos(delta_4-
phiY_4_3-delta_3) + Y_4_5*V_5*cos(delta_4-phiY_4_5-
delta_5) + Y_4_7*V_7*cos(delta_4-phiY_4_7-delta_7) +
Y_4_9*V_9*cos(delta_4-phiY_4_9-delta_9)) = P_4
-Y_4_4*V_4^2*sin(phiY_4_4) + V_4 * (Y_4_2*V_2*sin(
delta_4-phiY_4_2-delta_2) + Y_4_3*V_3*sin(delta_4-
phiY_4_3-delta_3) + Y_4_5*V_5*sin(delta_4-phiY_4_5-
delta_5) + Y_4_7*V_7*sin(delta_4-phiY_4_7-delta_7) +
Y_4_9*V_9*sin(delta_4-phiY_4_9-delta_9)) = Q_4
Y_5_5*V_5^2*cos(phiY_5_5) + V_5 * (Y_5_1*V_1*cos(
delta_5-phiY_5_1-delta_1) + Y_5_2*V_2*cos(delta_5-
phiY_5_2-delta_2) + Y_5_4*V_4*cos(delta_5-phiY_5_4-
delta_4) + Y_5_6*V_6*cos(delta_5-phiY_5_6-delta_6)) = P_5
-Y_5_5*V_5^2*sin(phiY_5_5) + V_5 * (Y_5_1*V_1*sin(
delta_5-phiY_5_1-delta_1) + Y_5_2*V_2*sin(delta_5-
phiY_5_2-delta_2) + Y_5_4*V_4*sin(delta_5-phiY_5_4-
delta_4) + Y_5_6*V_6*sin(delta_5-phiY_5_6-delta_6)) = Q_5
Y_6_6*V_6^2*cos(phiY_6_6) + V_6 * (Y_6_5*V_5*cos(
delta_6-phiY_6_5-delta_5) + Y_6_11*V_11*cos(delta_6-
phiY_6_11-delta_11) + Y_6_12*V_12*cos(delta_6-phiY_6_12-
delta_12) + Y_6_13*V_13*cos(delta_6-phiY_6_13-delta_13))
= P_6_g

```

```

V_6=V_6_sp
Y_7_7*V_7^2*cos(phiY_7_7) + V_7 * (Y_7_4*V_4*cos(
delta_7-phiY_7_4-delta_4) + Y_7_8*V_8*cos(delta_7-
phiY_7_8-delta_8) + Y_7_9*V_9*cos(delta_7-phiY_7_9-
delta_9)) = 0
-Y_7_7*V_7^2*sin(phiY_7_7) + V_7 * (Y_7_4*V_4*sin(
delta_7-phiY_7_4-delta_4) + Y_7_8*V_8*sin(delta_7-
phiY_7_8-delta_8) + Y_7_9*V_9*sin(delta_7-phiY_7_9-
delta_9)) = 0
Y_8_8*V_8^2*cos(phiY_8_8) + V_8 * (Y_8_7*V_7*cos(
delta_8-phiY_8_7-delta_7)) = 0
V_8=V_8_sp
Y_9_9*V_9^2*cos(phiY_9_9) + V_9 * (Y_9_4*V_4*cos(
delta_9-phiY_9_4-delta_4) + Y_9_7*V_7*cos(delta_9-
phiY_9_7-delta_7) + Y_9_10*V_10*cos(delta_9-phiY_9_10-
delta_10) + Y_9_14*V_14*cos(delta_9-phiY_9_14-delta_14))
= P_9
-Y_9_9*V_9^2*sin(phiY_9_9) + V_9 * (Y_9_4*V_4*sin(
delta_9-phiY_9_4-delta_4) + Y_9_7*V_7*sin(delta_9-
phiY_9_7-delta_7) + Y_9_10*V_10*sin(delta_9-phiY_9_10-
delta_10) + Y_9_14*V_14*sin(delta_9-phiY_9_14-delta_14))
= Q_9
Y_10_10*V_10^2*cos(phiY_10_10) + V_10 * (Y_10_9*V_9*
cos(delta_10-phiY_10_9-delta_9) + Y_10_11*V_11*cos(
delta_10-phiY_10_11-delta_11)) = P_10
-Y_10_10*V_10^2*sin(phiY_10_10) + V_10 * (Y_10_9*V_9
*sin(delta_10-phiY_10_9-delta_9) + Y_10_11*V_11*sin(
delta_10-phiY_10_11-delta_11)) = Q_10
Y_11_11*V_11^2*cos(phiY_11_11) + V_11 * (Y_11_6*V_6*
cos(delta_11-phiY_11_6-delta_6) + Y_11_10*V_10*cos(
delta_11-phiY_11_10-delta_10)) = P_11
-Y_11_11*V_11^2*sin(phiY_11_11) + V_11 * (Y_11_6*V_6
*sin(delta_11-phiY_11_6-delta_6) + Y_11_10*V_10*sin(
delta_11-phiY_11_10-delta_10)) = Q_11
Y_12_12*V_12^2*cos(phiY_12_12) + V_12 * (Y_12_6*V_6*
cos(delta_12-phiY_12_6-delta_6) + Y_12_13*V_13*cos(
delta_12-phiY_12_13-delta_13)) = P_12
-Y_12_12*V_12^2*sin(phiY_12_12) + V_12 * (Y_12_6*V_6
*sin(delta_12-phiY_12_6-delta_6) + Y_12_13*V_13*sin(
delta_12-phiY_12_13-delta_13)) = Q_12
Y_13_13*V_13^2*cos(phiY_13_13) + V_13 * (Y_13_6*V_6*
cos(delta_13-phiY_13_6-delta_6) + Y_13_12*V_12*cos(
delta_13-phiY_13_12-delta_12) + Y_13_14*V_14*cos(delta_13
-phiY_13_14-delta_14)) = P_13
-Y_13_13*V_13^2*sin(phiY_13_13) + V_13 * (Y_13_6*V_6
*sin(delta_13-phiY_13_6-delta_6) + Y_13_12*V_12*sin(
delta_13-phiY_13_12-delta_12) + Y_13_14*V_14*sin(delta_13
-phiY_13_14-delta_14)) = Q_13
Y_14_14*V_14^2*cos(phiY_14_14) + V_14 * (Y_14_9*V_9*
cos(delta_14-phiY_14_9-delta_9) + Y_14_13*V_13*cos(

```

```

delta_14-phiY_14_13-delta_13)) = P_14
      -Y_14_14*V_14^2*sin(phiY_14_14) + V_14 * (Y_14_9*V_9
      *sin(delta_14-phiY_14_9-delta_9) + Y_14_13*V_13*sin(
      delta_14-phiY_14_13-delta_13)) = Q_14
end

```

6 Zaključak

U okviru ovog zadatka uspješno je projektovan i implementiran softverski konvertor tokova snaga iz .m formata u .dmodl format modela. Kroz realizaciju zadatka su povezani teorijski koncepti i softverska implementacija.

Primjenom programskog jezika C++ i korištenjem multithreaded arhitekture, procesi parsiranja i matematičkog generisanja jednačina u polarnim koordinatama su odvojeni od grafičkog korisničkog interface-a, čime je obezbijedena stabilnost i rad aplikacije u realnom vremenu uz vizuelni prikaz napretka konverzije. Implementacija cross-platform podrške kroz CMake obezbjeđuje prenosivost rješenja, dok integracija natID sparse solver modula minimizira potrebu zauzimanja memorije.

Konačni rezultati testiranja na MATPOWER modelima standardnih mreža potvrđuju da generisani modeli omogućavaju dTwin okruženju stabilnu i brzu konvergenciju ka naponima čvorova, čineći ovaj plugin funkcionalnim alatom u elektroenergetskoj analizi.

Literatura

- [1] Predavanja iz predmeta *Strukture i režimi rada elektroenergetskih sistema*, Elektrotehnički fakultet, Univerzitet u Sarajevu, 2026.
- [2] *natID*, I. Džafić, GitHub repozitorij, dostupno na: <https://github.com/idzafic/natID>
- [3] *dTwin*, I. Džafić, GitHub repozitorij, dostupno na: <https://github.com/idzafic/dTwin>
- [4] *MATPOWER*, dostupno na: <https://matpower.org/>